



CHENCHU'S

Interview Series

AZURE DATA FACTORY (ADF)

Real time Interview Question & Answers

Part-01

1. How do you schedule pipelines in ADF?

To schedule pipelines in Azure Data Factory (ADF), use triggers.

There are three main types:

Schedule Trigger (for specific intervals like daily or hourly),

Tumbling Window Trigger (for fixed, non-overlapping windows),

Event Trigger (based on events like file creation in Blob Storage).

Create a trigger in the Author tab, configure it with the appropriate settings, and attach it to your pipeline. Finally, publish the pipeline to activate the schedule. Monitor and manage triggers in the Monitor and Manage sections.

2.What are the different integration runtimes available in Azure Data Factory, and when would you use each?

ADF offers three types of integration runtimes:

- 1.Azure Integration Runtime (for cloud data movement, transformations).
 2. Self-hosted Integration Runtime (for on-premises data movement)
 3. Azure-SSIS Integration Runtime (for running SSIS packages)
- - **Example:** For moving data from on-premises SQL Server to Azure Blob Storage, you'd use a Self-hosted Integration Runtime.

3.How do you implement data flow transformations in Azure Data Factory? Can you walk through a specific transformation you've used?

To implement data flow transformations in Azure Data Factory (ADF), first create a Mapping Data Flow under the Author tab. Add a Source transformation to import data, then apply transformations like Filter, Aggregate, Join, or Derived Column to manipulate the data.

For example, in a Derived Column transformation, you can create a new column by applying expressions to existing columns (e.g., concatenating first and last names).

Once the transformations are defined, add a Sink to load the transformed data to the destination, and finally, execute the data flow within a pipeline.

4.How do you handle schema drift in Azure Data Factory?

In Azure Data Factory (ADF), schema drift is handled using Mapping Data Flows with the Allow Schema Drift option enabled. This allows dynamic handling of changes in the schema (e.g., added or removed columns) without manual intervention.

The Source transformation detects schema changes, and the Auto Mapping feature in Sink maps columns automatically.

You can also use the Select transformation to explicitly choose or remove columns dynamically. This makes the data flow adaptable to evolving schemas across different sources.

5. How do you automate a stored procedure in an ADF pipeline?

To automate a stored procedure in an ADF pipeline, use the Stored Procedure Activity. Add this activity to your pipeline, configure it by selecting a linked service connected to your database, and specify the stored procedure name. If needed, pass parameters to the procedure. Finally, schedule the pipeline using triggers and publish it to automate execution.

6. How do you integrate ADF with CI/CD?

To integrate Azure Data Factory (ADF) with CI/CD, follow these steps:

1. **Connect ADF to Git:** In ADF, link a Git repository (e.g., Azure Repos or GitHub) for version control.
2. **Create Branches:** Use feature branches for development and merge changes into the main branch when ready.
3. **Build Pipeline:** In Azure DevOps, create a build pipeline to validate and store ADF artifacts (JSON files).
4. **Release Pipeline:** Set up a release pipeline to deploy ADF artifacts to other environments (e.g., test or production).
5. **Automate with Triggers:** Use triggers in Azure DevOps to automate the build and release upon code changes.
6. **Monitor:** Track deployments and ensure changes are applied correctly across environments.

7. Describe how you would implement error handling and retry mechanisms in ADF pipelines.

- We can configure retry policies on activities and use the On Failure and On Completion conditions in pipelines to manage errors.
- **Example:** If a pipeline fails due to a timeout, the pipeline retries the activity, and if the retries fail, it sends an alert to the support team.

8. How can you use parameters and variables within ADF pipelines to make them more dynamic and reusable?

- Parameters allow passing values at runtime, making pipelines dynamic. Variables can be used for temporary storage of values within pipelines.
- **Example:** A pipeline could accept a source file path as a parameter and dynamically load data from different sources without modifying the pipeline structure.

9. What is the process of setting up logging and monitoring in ADF for tracking pipeline performance and failures?

- ADF provides built-in monitoring and diagnostic logging through the Monitor tab. You can configure Azure Monitor and Log Analytics to track pipeline runs, performance, and failures.
- **Example:** You can set up an alert for failed pipeline runs and capture logs for debugging purposes in Log Analytics.

10 Explain the concept of data lineage and how you would implement it in an Azure-based data pipeline.

- Data lineage refers to the tracking of data as it moves from source to destination, including all transformations. ADF can be integrated with Azure Purview to track and visualize lineage.
- **Example:** If data is extracted from SQL, transformed in ADF, and loaded into Azure Synapse, Purview can show the flow and transformations applied to the data.

11. Describe the process of implementing slowly changing dimensions (SCD) in Azure Data Factory.

To implement Slowly Changing Dimensions (SCD) in Azure Data Factory, we will follow these steps:

1. **Source Transformation:** Use a Source transformation to read the historical data from the dimension table.
2. **Lookup Transformation:** Implement a Lookup transformation to compare incoming records against the existing dimension table based on the business key.
3. **Alter Row:** Add an Alter Row transformation to mark rows as updates, inserts, or deletes using conditional logic.
4. **Surrogate Keys:** Create or update surrogate keys using a Surrogate Key transformation to assign unique identifiers to new records.
5. **Update Rows:** Use an Upsert action in the Sink to either insert new rows or update existing rows based on the output from the Alter Row **transformation**.
6. **Sink Transformation:** In the Sink transformation, configure the connection to your target dimension table.
7. **Historical Tracking:** Ensure to add fields like EffectiveDate, EndDate, and flags like **IsCurrent for versioning**.
8. **Schedule the Pipeline:** Finally, schedule the pipeline to periodically process incoming data.

12. How do you manage access control and security in ADLS?

- You can manage access to ADLS using Azure RBAC (Role-Based Access Control) and Access Control Lists (ACLs) for granular permissions on files and folders.
- **Example:** You can assign read-only access to certain folders within the data lake for business analysts while granting full access to data engineers.

13. Explain the lifecycle management of data in ADLS.

Data lifecycle management in Azure Data Lake Storage (ADLS) helps automate the process of storing, archiving, and deleting data efficiently. When data is first ingested, it's placed in a storage tier like Hot for frequently accessed information. As the data becomes less used over time, lifecycle policies can automatically move it to Cool or Archive tiers, which are cheaper but slower to access. You can set rules to define when data should be moved between these tiers based on how old it is or how often it's accessed. Retention policies can also be applied to automatically delete data after a certain period, helping manage storage costs and ensure compliance with data retention regulations. ADLS supports encryption to keep data secure, and versioning allows you to keep track of changes to files. Finally, after its useful life, the data is permanently deleted, ensuring efficient use of storage resources.

14. How would you optimize data partitioning in ADLS for performance?

Optimizing data partitioning in Azure Data Lake Storage (ADLS) for performance involves structuring your data in a way that improves query speed and reduces costs. The key strategies:

1. **Partition by Relevant Columns:** Partition data based on frequently queried columns, like date, region, or category. This allows queries to target specific subsets of data, reducing the amount of data scanned.
2. **Use Hierarchical Folder Structures:** Organize your data in a folder hierarchy that mirrors your partitioning strategy (e.g., /year=2024/month=10/region=US). This helps tools like Azure Data Factory (ADF) and Azure Databricks efficiently access relevant partitions.
3. **Optimize File Sizes:** Aim for optimal file sizes (usually between 100 MB and 1 GB) to balance between the overhead of handling small files and the inefficiency of very large files. This ensures better parallelism during processing.
4. **Avoid Over-Partitioning:** Avoid creating too many small partitions, as it can lead to excessive metadata management and longer query times. Strike a balance between granularity and performance.
5. **Leverage Partition Pruning:** Use query tools that support partition pruning, like Spark or Synapse, which skip unnecessary partitions based on query filters, improving query speed.
6. **Compression:** Apply columnar file formats like Parquet or ORC with compression (e.g., Snappy) to reduce storage costs and speed up data scans.
7. **Regular Data Reorganization:** Periodically reorganize partitions to merge small files and optimize query performance as data grows over time.

These strategies will help ensure that your data in ADLS is efficiently partitioned for both storage and performance optimization.

15. What are some best practices for optimizing performance in Azure Data Factory pipelines?

Some of best practices for optimizing performance in Azure Data Factory (ADF) pipelines:

1. **Use Data Flow Debug Mode Efficiently:** Limit data flow debug sessions to essential testing only, as it consumes resources. Disable debug when not needed.
2. **Optimize Data Movement:** Use Copy Activity with staging in Azure Blob or Azure Data Lake for large datasets, enabling parallelism and compression.
3. **Leverage Parallelism:** Increase the degree of parallelism in pipeline settings and within copy activities to maximize resource usage and speed up processing.
4. **Filter Data Early:** Apply filters as early as possible in data flows or transformations to reduce data volume.
5. **Use Partitioning:** For large datasets, partition your source data in data flows or copy activities to optimize performance, especially with SQL Server or Blob Storage.
6. **Monitor and Auto-Scale IRs:** Use Auto-Scaling Integration Runtimes (IR) to adjust resources dynamically based on workload needs.
7. **Minimize Data Movement:** Whenever possible, avoid unnecessary data movement between services or regions by keeping data processing close to the source.

These practices help reduce latency, improve throughput, and maximize resource efficiency in ADF pipelines.

16. How would you handle incremental data loading in ADF from a SQL database?

Steps for Incremental Loading with Watermark:

1. **Add a Timestamp Column:** Ensure your source table includes a timestamp column (e.g., LastModifiedDate) that tracks when rows are inserted or updated.
2. **Set Up a Watermark Table:** Create a watermark table in your database to store the LastLoadTime (i.e., the last successful load's timestamp).
3. **Create a Pipeline:** In ADF, create a pipeline with a Copy Activity or Mapping Data Flow.
4. **Use a Parameter for Last Load Time:** Use a Lookup Activity to fetch the LastLoadTime from the watermark table and store it as a pipeline parameter (e.g., @activity('LookupWatermark').output.firstRow.LastLoadTime).
5. **Filter Data Using Watermark:** In the Source Dataset, use a query like `SELECT * FROM Table WHERE LastModifiedDate > @LastLoadTime` to fetch only new or updated records.
6. **Update the Watermark:** After the data load, use a Stored Procedure Activity or another Copy Activity to update the watermark table with the current timestamp of the latest loaded record.
7. **Schedule the Pipeline:** Automate the pipeline using a trigger to run on a regular basis (e.g., hourly, daily).

This method ensures you load only the incremental changes using the watermark stored in the database.

17. How do you implement pipeline dependencies and parallel execution in ADF to ensure efficient data processing workflows?

To implement pipeline dependencies and parallel execution in ADF:

1. Use On-Success, On-Failure, and On-Completion conditions between activities to manage dependencies. For example, run a transformation step only if data extraction succeeds.
 2. Use the Wait Activity to pause execution until a condition is met (e.g., waiting for a file to land before processing).
 3. For parallel execution, set Max Concurrent Activities to enable multiple activities to run simultaneously.
 4. In ForEach Activity, you can set the Batch Count to execute multiple iterations (e.g., files) in parallel.
For example, if processing 10 files, setting Batch Count to 5 will process 5 files simultaneously.
 5. Data Flow allows parallelism by configuring partitioning for better performance during data transformations.
 6. In cases where you need multiple pipelines to execute one after another, use the Pipeline Activity to trigger dependent pipelines.
 7. Ensure optimal concurrency in Copy Activity for large datasets by configuring the parallel copy settings.
 8. Use monitoring in the ADF UI to track pipeline performance and adjust concurrency as necessary.
 9. For pipeline failure, configure On-Failure dependencies to handle retries or error notifications.
 10. Implement data validation steps in parallel to ensure data quality across sources.
For example, in a sales data pipeline, process regional data in parallel, then trigger a report when all regions are processed.
- Combining these techniques ensures efficient and scalable data workflows in ADF.

18.Can you describe a scenario where data pipeline monitoring failed to catch an issue? How did you detect and resolve the problem?

Scenario: Silent Data Corruption in Financial Reports

Problem:

An organization had an ADF pipeline that ingested daily transaction data from multiple sources into a centralized Azure SQL Data Warehouse. The pipeline was designed to process data without any failures, and ADF monitoring indicated successful executions. However, business analysts discovered discrepancies in financial reports, suggesting missing or incorrect data.

Cause:

Schema Changes in the Source System: One of the source systems changed its schema (e.g., adding new columns and changing column types), but the ADF pipeline was still extracting data based on the old schema. This led to missing or incorrect data being processed silently.

Incorrect Data Mapping: The pipeline's transformations weren't updated to reflect the schema changes, causing critical fields to be skipped or mapped incorrectly during the data flow.

Detection:

The issue was discovered when business analysts noticed anomalies in financial reports, such as missing transactions. A manual audit of the source data versus the ingested data uncovered missing columns and incorrect mappings in the pipeline.

Resolution:

Schema Drift: The pipeline was modified to handle schema drift, allowing it to detect new columns automatically. In ADF's Mapping Data Flow, the Allow Schema Drift option was enabled, ensuring the pipeline adapts to changes without manual intervention.

Data Validation Layer: A new data validation step was added at the end of the pipeline, checking row counts and data completeness before loading to the final destination. Custom checks were added to compare source and target row counts and critical column values.

Monitoring Enhancements: Custom alerts were set up using Azure Monitor to catch data inconsistencies. Row counts, null checks, and other integrity checks were introduced to detect any silent failures or incomplete data.

Lessons Learned:

Silent Failures: Even when ADF pipelines show success in the monitoring tool, silent data issues can occur, particularly when schema changes are involved.

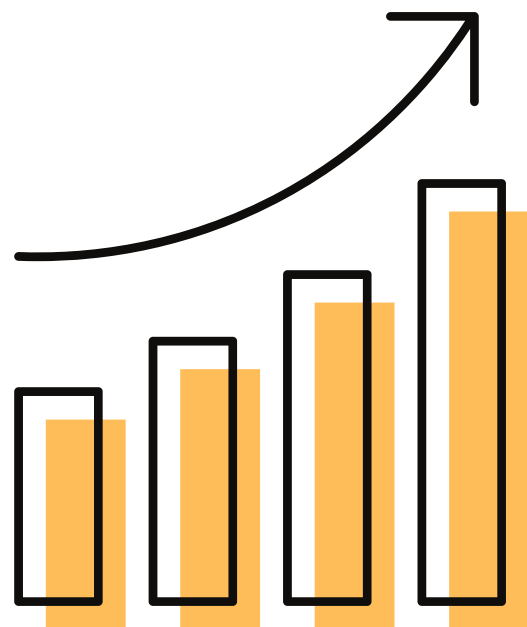
Proactive Monitoring: Implementing custom validations and schema drift handling helps ensure the integrity of the data, even when ADF's built-in monitoring reports successful pipeline execution.



ANIL REDDY CHENCHU



***Torture** the data, and it will confess to anything*



DATA ANALYTICS



Azure
Synapse
Analytics



Happy Learning 😊

SHARE IF YOU LIKE THE POST

Lets Connect to discuss more on Data



www.linkedin.com/in/chenchuanil